# Containerisation and reproducibility

**How to use containers and workflows to make your work more reproducible**

ASI WA-branch workshop 9th July 2026

Dr Aaron B Beasley

# By the end of today you will be able to:

- ✓ Initialise Docker and install containers
- ✓ Initialise Apptainer and install containers
- ✓ Initialise Nextflow workflows

docker APPTAINER nextflow

# Why containerisation?

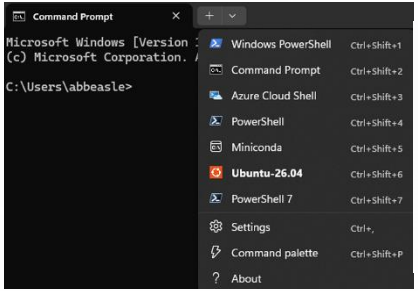
- Isolation of many operating system packages
  - Kernel is shared
  - User space is packaged
- Self-sufficiency
  - Anything needed is contained (e.g. anything outside of R or python libraries).
- Docker tries to isolate from the host
- Apptainer borrows more from the host

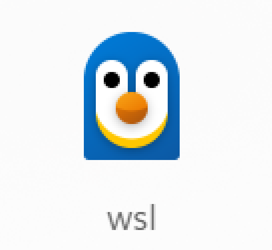
- Docker is good for:
  - Personal PC
  - More strict isolation

- Apptainer is good for:
  - HPC space – no root permissions!

# Windows users: get WSL2

- ➢ Load terminal and load a new PowerShell tab




- ➢ Install WSL2


- ➢ By default this will install *Ubuntu*
- ➢ Distro can be changed with –d
- ➢ wsl.exe --list --online
- ➢ You may need to reset your PC

# Mac users:

- ➢ Get homebrew

/bin/bash -c "$(curl -fsSL [https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh](https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh))“

- ➢ Install lima

brew install lima

- ➢ Create an instance

limactl create --name=ubuntu-vm template://ubuntu-lts

limactl start ubuntu-vm

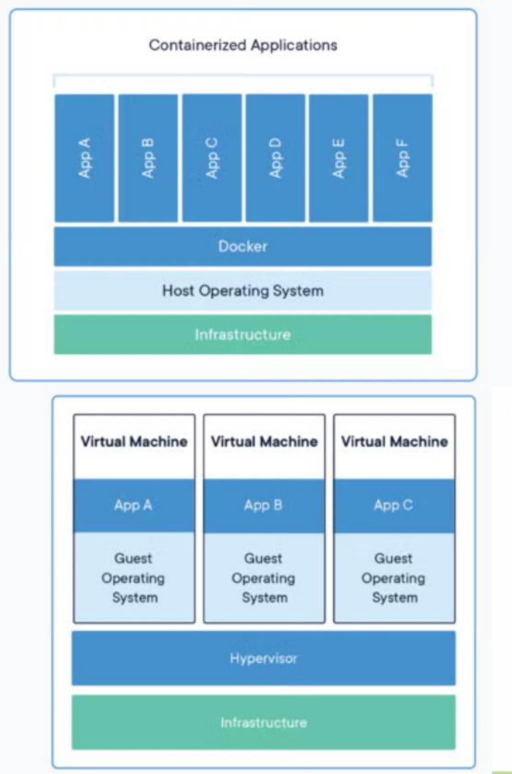
- ➢ Boot the VM

limactl shell ubuntu-vm
Pass through —cpu and —memory
As needed

**NOTE: you can also do**
limactl start
template://apptainer or
template:docker
**But to keep consistent we use the ubuntu vm here**

# Starting point: Using Docker

- What is docker?
  - Used to create containers – an executable that contains everything to reproduce an analysis… "it works on my machine".
  - Docker images are read-only and serve as a template for creating a container
  - Docker runs securely and in isolated – allows you to run what ever version of software you need without worrying about versioning
  - Faster than spooling up a VM
  - Portable – can run anywhere (that has docker installed)



# Starting point: Using Docker

- CLI first (Debian/Ubuntu example):
- **Docker instructions very easy to follow! Just copy/paste their code.**
- https://docs.docker.com/engine/install/ubuntu/

For docker you need SUPERUSER/ROOT access.

- **The steps involve:**
  1. Removing old versions
  2. Updating apt (the local package index)
  3. Installs ca-certificates and curl (if not already present)
  4. Creates a folder to store keys + makes the permissions owner read/write/execute but others only read/execute
  5. Downloads the docker security key and saves it to the previous path
  6. Adds the docker repository to apt source list
  7. Refreshes package lists
  8. Installs docker packages needed – specific versions can be chosen → start from the beginning to uninstall any old versions
  9. Check if working!

# Starting point: Using Docker

- The docker daemon is owned by root
- You can set Docker to run on users without elevating privileges with sudo… this is only advisable on your personal computer. This will also never occur on large clusters due to security
- I do this for my personal computer
  1. sudo groupadd docker
  2. sudo usermod -aG docker $USER
  3. newgrp docker
  4. groups ← you should see "docker" → if not reload terminal
  5. docker run hello-world → should run without sudo

# Starting point: Using Docker

- To use the CLI

➢ docker pull rocker/tidyverse:4.5

➢ docker run -d -p 8787:8787 -e PASSWORD=<password> --name r_tidy rocker/tidyverse:4.5

Where =<password> can be whatever you want

➢ docker ps

➢ Navigate to [http://localhost:8787](http://localhost:8787)

➢ Enter rstudio and password!

# Starting point: Using Docker

- Alternately, you can just push a script through the container

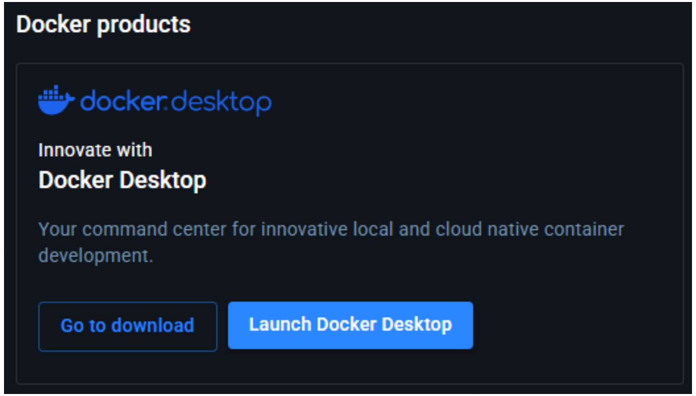
➢ docker run --rm \

```
-v /path/to/my/data/dir:data_dir \
-w /data_dir \
rocker/tidyverse:4.5 \
Rscript my_cool_rscript.R
```

- This will execute the script within the environment, then delete the environment after (--rm, saves space in exchange for time).

# Starting point: Using Docker Desktop

- Go to https://app.docker.com/
- Make an account / login with SSO
- Download docker:desktop

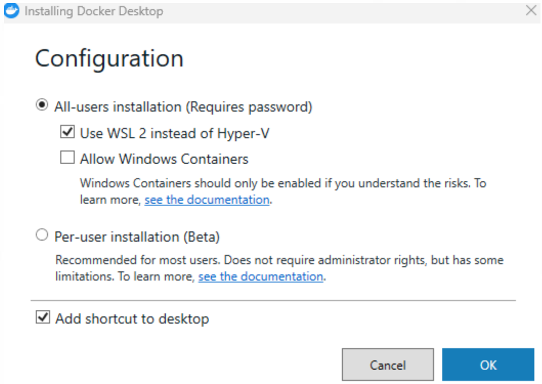


# Starting point: Using Docker Desktop

**Windows:**

Run .exe

Follow instructions



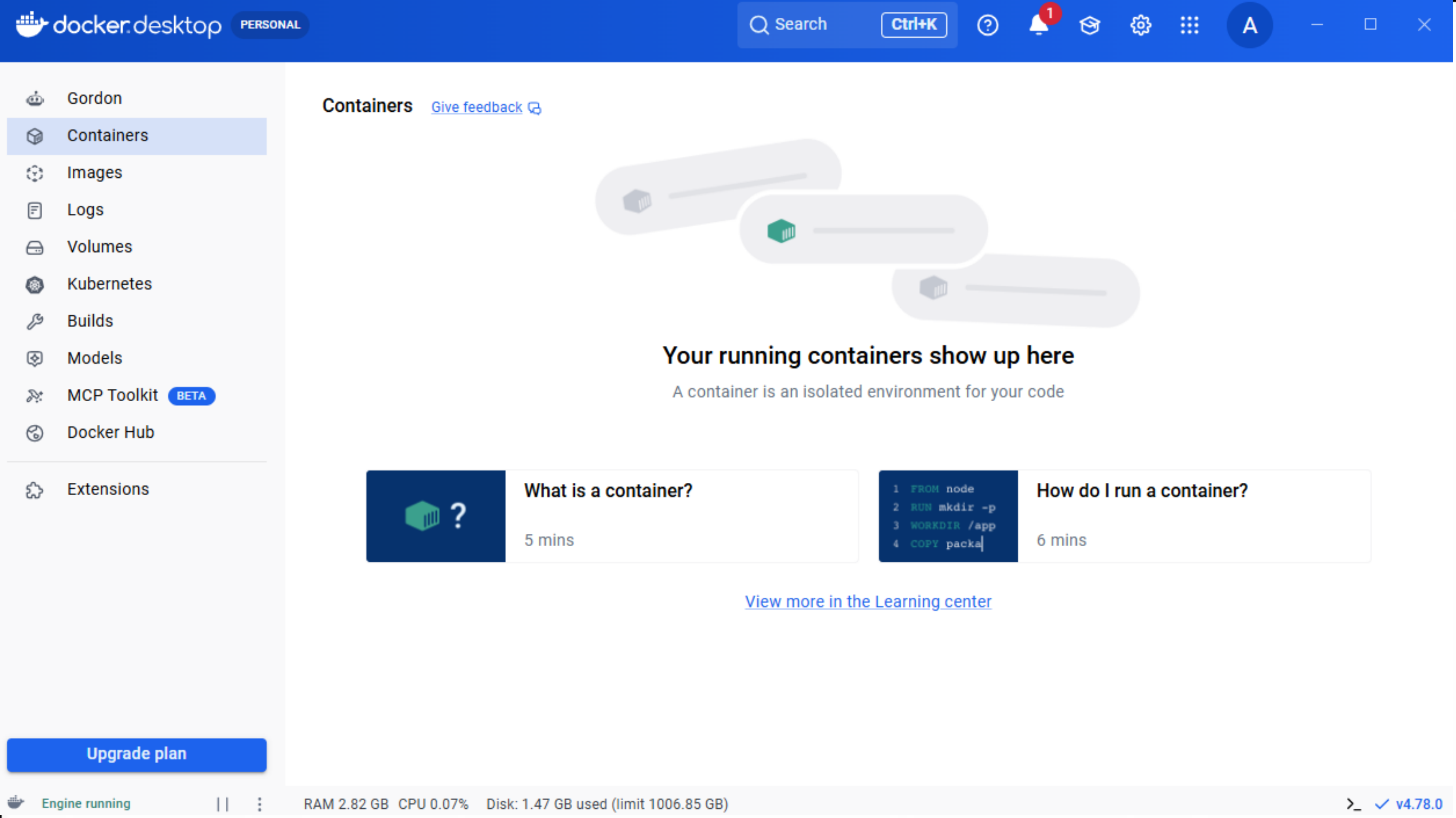

**Mac:**

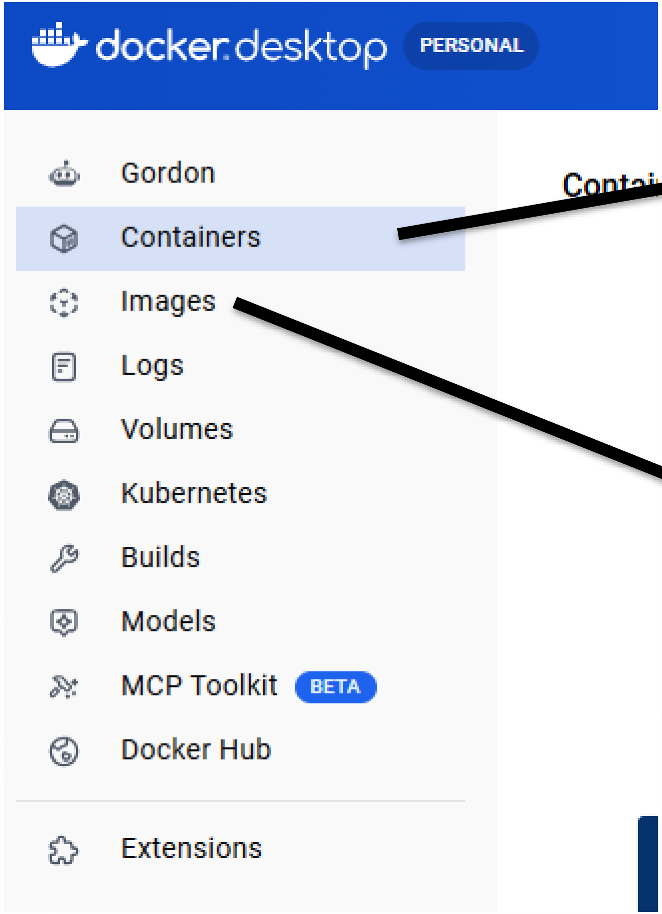
In terminal:

softwareupdate --install-rosetta

Run .dmg file

Follow instructions

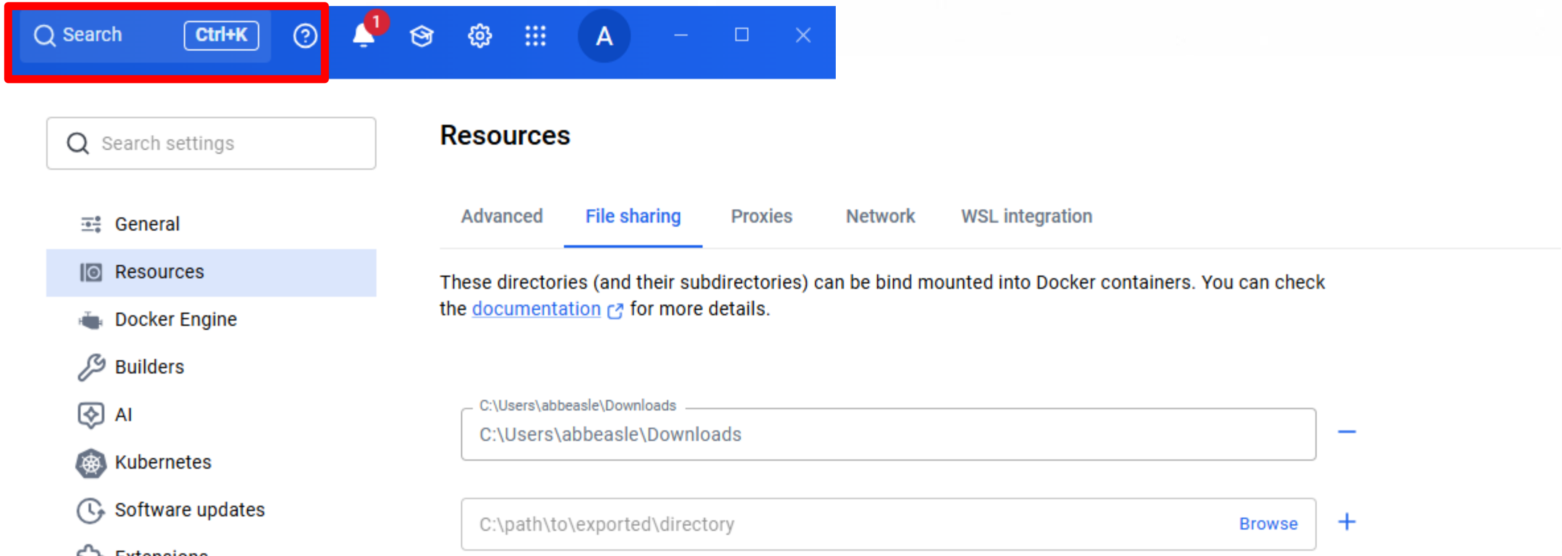

docker desktop
PERSONAL
Search
Ctrl+K
1
A
Gordon
Containers
Images
Logs
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Extensions
Upgrade plan
Engine running
Containers
Give feedback
Your running containers show up here
A container is an isolated environment for your code
What is a container?
5 mins
1 FROM node
2 RUN mkdir -p
3 WORKDIR /app
4 COPY packa
How do I run a container?
6 mins
View more in the Learning center
RAM 2.82 GB CPU 0.07% Disk: 1.47 GB used (limit 1006.85 GB)
v4.78.0



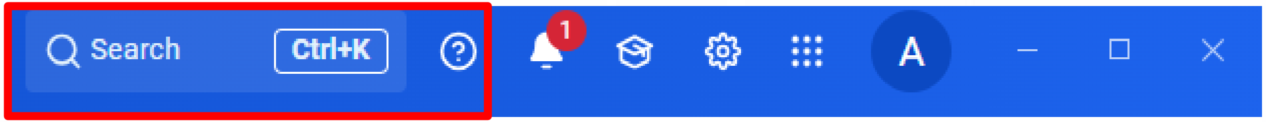

The container is the runnable process of an image and is mutable

Immutable package containing everything needed (you pull these from docker or create them)

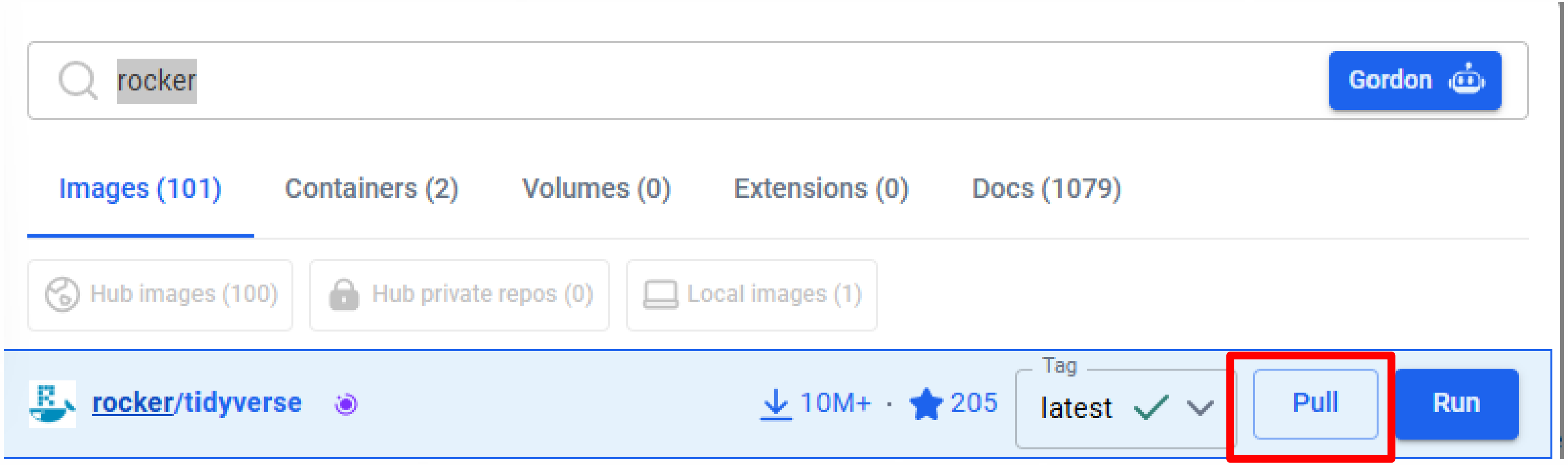
# Let's grab an R image ...



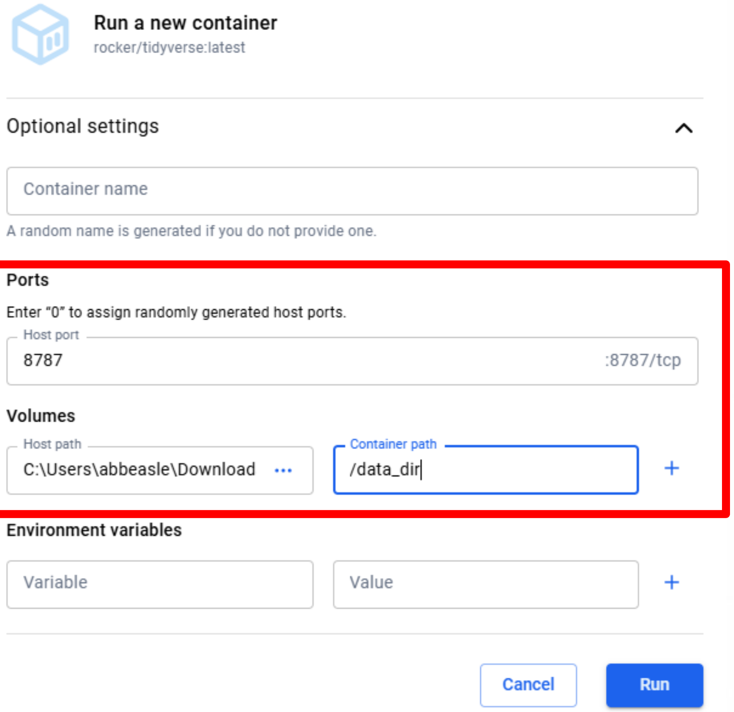
# Let's grab an R image ...




# Search rocker/tidyverse and pull the image

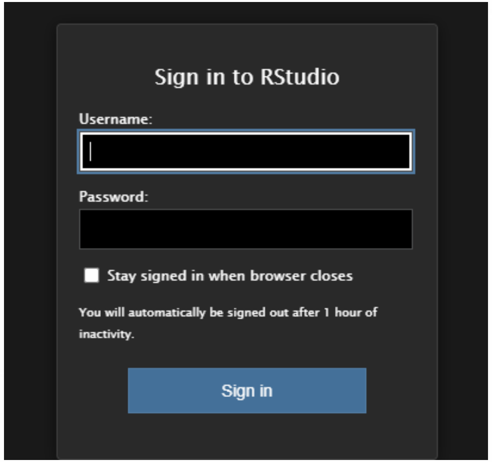


# Lets set rstudio port to its default 8787
# And pass through a volume

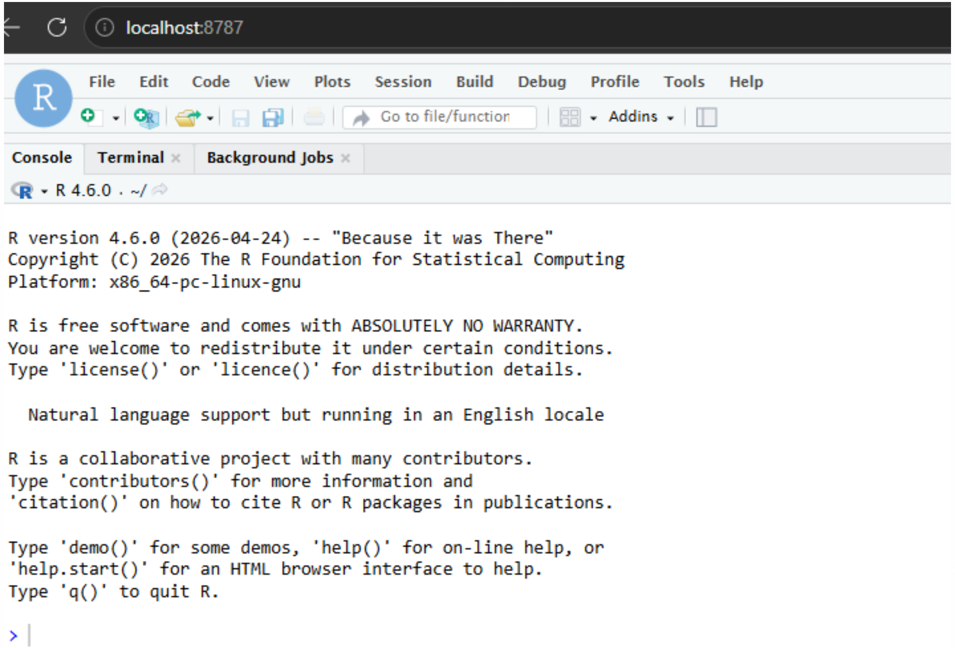


# Rstudio with is now active on localhost:8787
# Default login is rstudio
# Password is …

```
[s6-init] making user provided files available at /var/run/s6/etc...exited 0.
[s6-init] ensuring user provided files have correct perms...exited 0.
[fix-attrs.d] applying ownership & permissions fixes...
[fix-attrs.d] done.
[cont-init.d] executing container initialization scripts...
[cont-init.d] 01_set_env: executing...
skipping /var/run/s6/container_environment/HOME
skipping /var/run/s6/container_environment/RSTUDIO_VERSION
[cont-init.d] 01_set_env: exited 0.
[cont-init.d] 02_userconf: executing...

tput: No value for $TERM and no -T specified
The password is set to eis3Eeyei1kae6Uu
If you want to set your own password, set the PASSWORD environment variable. e.g. run with:
docker run -e PASSWORD=<YOUR_PASS> -p 8787:8787 rocker/rstudio
tput: No value for $TERM and no -T specified

[cont-init.d] 02_userconf: exited 0.
[cont-init.d] done.
[services.d] starting services
[services.d] done.
```



# Congratulations – you are now running R in a container!



# What is Apptainer

- Similar to Docker, but:
  - Is rootless – runs in user space
  - All dependencies are in a single file ".sif"
  - HPC friendly
  - Can pull docker images directly from docker hub and convert them to .sif

# Installing Apptainer

- You must be using linux
  - ➢ sudo apt update
  - ➢ sudo apt install -y software-properties-common
  - ➢ sudo add-apt-repository -y ppa:apptainer/ppa
  - ➢ sudo apt update
  - ➢ sudo apt install -y apptainer

# Pulling an image Apptainer

- Apptainer can be used to run Apptainer containers or even those supporting OCI such as *Docker*
- If pulling from *Docker Hub, it is recommended to pull the file rather than run as to not generate too many API requests*
- *It can be time consuming and resource heavy to build a sif file*. *For times sake we will grab small containers*.

➢ Lets get the lolcow

```
apptainer pull docker://sylabsio/lolcow
```

# Using Apptainer

- Running the .sif file (default run)
  - apptainer run lolcow_latest.sif
  - Or ./lolcow_latest.sif
- Execute a command from the container
  - apptainer exec lolcow_latest.sif cowsay “Hi”
- Open the container shell
  - apptainer shell lolcow_latest.sif
  - cowsay “hi”


# What is NextFlow

- Nextflow unlike Docker and Apptainer is not a container
- It is a workflow system
- It can utilise containers or other execution options as “modules”
- It is a scripting language → this is only important if you want to write your own NextFlow pipelines (not covered today)
- NextFlow will provide abstraction between the script logic and the execution

Cutdown NextFlow example:

```
// Script parameters
params.query = "/some/data/sample.fa"
params.db = "/some/path/pdb"

// Define the blast_search process
process blast_search {
input:
path query
path db

output:
path "top_hits.txt"

script:
"""
blastp -db $db -query $query -outfmt 6 > blast_result
cat blast_result | head -n 10 | cut -f 2 > top_hits.txt
"""
}
// Define the workflow
workflow {
def query_ch = channel.fromPath(params.query)
blast_search(query_ch, params.db)
}
```

# Installing NextFlow

- While you can install within Host, I recommend conda as you may need different system java versions, for example.

> **⚠ WARNING**
>
> Installing Nextflow via Conda may lead to outdated versions, dependency conflicts, and Java compatibility issues. Using the self-installing package is recommended for a more reliable and up-to-date installation.

- I have not ever encountered issues, but if there ever arises, you can always install the software on Host

➢ conda! (or mamba, or micromamba)
  - ➢ conda create -n goodenvname bioconda::nextflow
  - ➢ conda activate goodenvname
  - ➢ nextflow info

# Pulling a workflow

- Nf-core hosts many nice workflows for routine tasks
- First run the test to make sure your env is working ok
- nextflow run nf-core/demo -profile test,<container_of_choice> --outdir results
- <container_of_choice>
    - Replace with docker, apptainer, or conda to your wish e.g.
    - *nextflow run nf-core/demo -profile test,docker --outdir results*
    - I would use docker unless on a restricted HPC – I find it is quicker to initialise and a little less finicky

# Success!

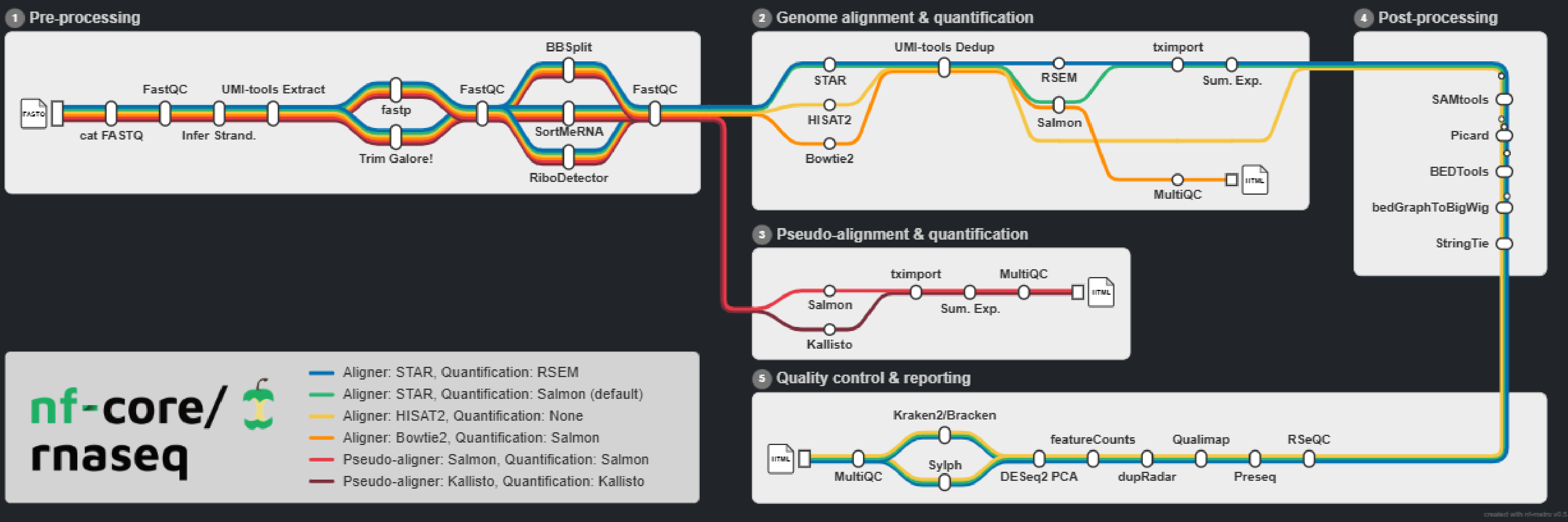
```
!! Only displaying parameters that differ from the pipeline defaults !!
-----------------------------------------------------

* The pipeline
    https://doi.org/10.5281/zenodo.12192442

* The nf-core framework
    https://doi.org/10.1038/s41587-020-0439-x

* Software dependencies
    https://github.com/nf-core/demo/blob/master/CITATIONS.md

executor >  local (8)
[6a/35f29c] NFCORE_DEMO:DEMO:FASTQC (SAMPLE1_PE)     [100%] 3 of 3 ✔
[fe/29d6e9] NFCORE_DEMO:DEMO:SEQTK_TRIM (SAMPLE1_PE) [100%] 3 of 3 ✔
[c5/98376a] NFCORE_DEMO:DEMO:COWPY                   [100%] 1 of 1 ✔
[9a/e9c162] NFCORE_DEMO:DEMO:MULTIQC (demo)          [100%] 1 of 1 ✔
-[nf-core/demo] Pipeline completed successfully-
Completed at: 24-Jun-2026 23:32:27
Duration    : 2m 16s
CPU hours   : (a few seconds)
Succeeded   : 8
```

# Nf-core has many well-tested pipelines

## Pipelines

Browse the 152 pipelines that are currently available as part of nf-core.

# Such as the nf-core rnaseq workflow



# Nf-core also has a nifty pipeline launcher

## >_ Input/output options

Define where the pipeline should find input data and save output data.

`--input` *

Path to the sample sheet (CSV) containing metadata about the experimental samples.

Provide the full path to a comma-separated sample sheet with 4 columns and a header row. This file is required to run the pipeline. See the nf-core/rnaseq sample sheet documentation for example format.

`--outdir` *

The output directory where the results will be saved. You have to use absolute paths to storage on Cloud infrastructure.

`--email`

Email address for completion summary.

`--multiqc_title`

MultiQC report title. Printed as page header, used for filename if not otherwise specified.